Sri Sivasubramaniya Nadar College of Engineering, Chennai
(An Autonomous Institution Affiliated to Anna University)

Degree & Branch	B.E. Computer Science & Engineering	Semester	VI
Subject Code & Name	UCS2612 – Machine Learning Algorithms Laboratory		
Academic Year	2025–2026 (Even)	Batch	2023–2027
Due Date	29-03-2026		
Name	Priya Verma - 3122235001102		

Experiment 8

Effect of PCA on Regression and Classification Models

1 Objective

To study the impact of **Principal Component Analysis (PCA)** on the performance of machine learning models.

Students must:

a) Train models in the original feature space (Without PCA)

b) Train models in reduced feature space (With PCA)

c) Compare performance using 5-fold cross-validation

2 Dataset

Dataset: **PCA_Full_Lab_Dataset_1000_Samples.csv**

Features: Academic and performance-related attributes **Regression Target:** Final_Score_Regression **Classification Target:** Performance_Level_Classification

3 Implementation Details

The following code snippets illustrate the core implementation of the classifiers:

Important Libraries to be included

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

from sklearn.model_selection import cross_val_score, KFold, GridSearchCV
from sklearn.preprocessing import StandardScaler, LabelEncoder
```

```python
from sklearn.decomposition import PCA

from sklearn.linear_model import LinearRegression, LogisticRegression
from sklearn.ensemble import RandomForestRegressor
from sklearn.svm import SVC

from sklearn.metrics import make_scorer, mean_squared_error, r2_score,
    accuracy_score, f1_score

plt.rcParams.update({
    "font.family": "Arial",
    "font.size": 15,
    "axes.titlesize": 15,
    "axes.titleweight": "bold",
    "axes.labelsize": 15,
    "axes.labelweight": "bold",
    "xtick.labelsize": 13,
    "ytick.labelsize": 13
})
```

1. Load the dataset

```python
# Load dataset
df = pd.read_csv("Dataset.csv")

print("Dataset Shape:", df.shape)
df.head()
# Number of samples and features
print("Number of samples:", df.shape[0])
print("Number of features:", df.shape[1]-2)

# Target distribution
print("\nClassification Target Distribution")
print(df['Performance_Level_Classification'].value_counts())
```

2. Preprocessing Steps

```python
# 2a. Identify target columns
# Adjust these column names if your CSV uses different names
REGRESSION_TARGET    = "Final Score"        # continuous target
CLASSIFICATION_TARGET = "Performance Level" # categorical target

# Auto-detect if column names differ
all_cols = df.columns.tolist()
if REGRESSION_TARGET not in all_cols:
    # Try to find a numeric column that could be regression target
    num_cols = df.select_dtypes(include=[np.number]).columns.tolist()
    REGRESSION_TARGET = num_cols[-1]
    print(f"[Auto-detected] Regression target    : '{REGRESSION_TARGET}'")

if CLASSIFICATION_TARGET not in all_cols:
    # Try to find a categorical/object column
    cat_cols = df.select_dtypes(include=["object", "category"]).columns.tolist()
    if cat_cols:
        CLASSIFICATION_TARGET = cat_cols[-1]
    else:
```

```
        # Fall back to binning the regression target
        CLASSIFICATION_TARGET = "Performance_Level_Binned"
        df[CLASSIFICATION_TARGET] = pd.qcut(
            df[REGRESSION_TARGET], q=3, labels=["Low", "Medium", "High"]
        )
    print(f"[Auto-detected] Classification target: '{CLASSIFICATION_TARGET}'")

print(f"\nRegression    target : '{REGRESSION_TARGET}'")
print(f"Classification target: '{CLASSIFICATION_TARGET}'")

# 2b. Separate feature columns
drop_cols = [REGRESSION_TARGET, CLASSIFICATION_TARGET]
drop_cols = [c for c in drop_cols if c in df.columns]
X_raw = df.drop(columns=drop_cols)

y_reg  = df[REGRESSION_TARGET].copy()
y_clf  = df[CLASSIFICATION_TARGET].copy()

# 2c. Handle missing values
missing_before = X_raw.isnull().sum().sum()
print(f"\nMissing values before imputation : {missing_before}")

# Numeric columns     median imputation
num_feat = X_raw.select_dtypes(include=[np.number]).columns
X_raw[num_feat] = X_raw[num_feat].fillna(X_raw[num_feat].median())

# Categorical columns     mode imputation then encode
cat_feat = X_raw.select_dtypes(include=["object", "category"]).columns
for col in cat_feat:
    X_raw[col] = X_raw[col].fillna(X_raw[col].mode()[0])
    X_raw[col] = LabelEncoder().fit_transform(X_raw[col].astype(str))

y_reg = y_reg.fillna(y_reg.median())

missing_after = X_raw.isnull().sum().sum()
print(f"Missing values after  imputation : {missing_after}")

# 2d. Encode classification target
le = LabelEncoder()
y_clf_enc = le.fit_transform(y_clf.astype(str))
print(f"\nClassification classes : {list(le.classes_)}")

# 2e. Dataset statistics
print(f"\nFeature matrix shape  : {X_raw.shape}")
print(f"\nRegression target distribution:")
print(f"  Mean={y_reg.mean():.2f}, Std={y_reg.std():.2f}, "
      f"Min={y_reg.min():.2f}, Max={y_reg.max():.2f}")
print(f"\nClassification target distribution:")
for cls, cnt in zip(le.classes_, np.bincount(y_clf_enc)):
    print(f"  {cls}: {cnt} samples ({100*cnt/len(y_clf_enc):.1f}%)")

# 2f. Standardize features
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X_raw)
X_scaled = pd.DataFrame(X_scaled, columns=X_raw.columns)
print(f"\nStandardization applied     m e a n {X_scaled.values.mean():.4f}, "
      f" s t d  {X_scaled.values.std():.4f}")
```

3. PCA Implementation

```
# Full PCA to study explained variance
pca_full = PCA(random_state=42)
pca_full.fit(X_scaled)

explained_var       = pca_full.explained_variance_ratio_
cumulative_var      = np.cumsum(explained_var)
n_components_95     = np.argmax(cumulative_var >= 0.95) + 1
n_features          = X_scaled.shape[1]

print(f"Total features                : {n_features}")
print(f"Components for 95% variance   : {n_components_95}")
print(f"Explained variance at 95% PC : {cumulative_var[n_components_95-1]*100:.2f
    }%")

# 3a. Scree Plot
fig, axes = plt.subplots(1, 2, figsize=(14, 5))

axes[0].bar(range(1, min(21, n_features+1)),
            explained_var[:20] * 100,
            color="steelblue", edgecolor="black", alpha=0.8)
axes[0].set_title("Scree Plot: Individual Explained Variance", fontweight="bold")
axes[0].set_xlabel("Principal Component")
axes[0].set_ylabel("Explained Variance (%)")
axes[0].grid(axis="y", alpha=0.4)

axes[1].plot(range(1, n_features+1), cumulative_var * 100,
             "bo-", markersize=4, linewidth=2)
axes[1].axhline(y=95, color="red", linestyle="--", linewidth=1.5, label="95%
    threshold")
axes[1].axvline(x=n_components_95, color="green", linestyle="--",
                linewidth=1.5, label=f"k={n_components_95} components")
axes[1].fill_between(range(1, n_features+1), cumulative_var * 100,
                     alpha=0.15, color="steelblue")
axes[1].set_title("Cumulative Explained Variance", fontweight="bold")
axes[1].set_xlabel("Number of Components")
axes[1].set_ylabel("Cumulative Variance (%)")
axes[1].legend()
axes[1].grid(alpha=0.4)

plt.suptitle("PCA Scree Plot Analysis", fontsize=14, fontweight="bold")
plt.tight_layout()
plt.savefig("pca_scree_plot.png", dpi=150)
plt.show()
print("Saved: pca_scree_plot.png")

# 3b. Apply PCA (retain 95% variance)
pca = PCA(n_components=n_components_95, random_state=42)
X_pca = pca.fit_transform(X_scaled)
X_pca = pd.DataFrame(X_pca, columns=[f"PC{i+1}" for i in range(n_components_95)])

print(f"\nTable 1: PCA Summary")
print(f"{'Components Chosen':<22} {n_components_95}")
print(f"{'Explained Variance (%)':<22} {cumulative_var[n_components_95-1]*100:.2f
    }%")
print(f"{'Justification':<22} Retain 95% of total variance with minimal
    information loss.")
```

```
X_no_pca = X_scaled   # Original scaled features
X_with_pca = X_pca    # PCA-reduced features

kf = KFold(n_splits=5, shuffle=True, random_state=42)

def cv_regression(model, X, y, kf):
    """Return per-fold MSE and R2."""
    mse_folds, r2_folds = [], []
    for train_idx, test_idx in kf.split(X):
        X_tr, X_te = X.iloc[train_idx], X.iloc[test_idx]
        y_tr, y_te = y.iloc[train_idx], y.iloc[test_idx]
        model.fit(X_tr, y_tr)
        y_pred = model.predict(X_te)
        mse_folds.append(mean_squared_error(y_te, y_pred))
        r2_folds.append(r2_score(y_te, y_pred))
    return np.array(mse_folds), np.array(r2_folds)

 def cv_classification(model, X, y, kf):
    """Return per-fold Accuracy and F1."""
    acc_folds, f1_folds = [], []
    for train_idx, test_idx in kf.split(X):
        X_tr, X_te = X.iloc[train_idx], X.iloc[test_idx]
        y_tr, y_te = y[train_idx], y[test_idx]
        model.fit(X_tr, y_tr)
        y_pred = model.predict(X_te)
        acc_folds.append(accuracy_score(y_te, y_pred))
        f1_folds.append(f1_score(y_te, y_pred, average="weighted", zero_division
   =0))
    return np.array(acc_folds), np.array(f1_folds)

y_reg_s  = pd.Series(y_reg.values)
y_clf_s  = np.array(y_clf_enc)
```

4. Regression Models

```
# 5a. Linear Regression
print("\n--- Linear Regression ---")
lr = LinearRegression()

lr_mse_no,  lr_r2_no  = cv_regression(lr, X_no_pca,   y_reg_s, kf)
lr_mse_pca, lr_r2_pca = cv_regression(lr, X_with_pca, y_reg_s, kf)

print(f"  No PCA      MSE: {lr_mse_no.mean():.4f}    {lr_mse_no.std():.4f}  "
      f"R  : {lr_r2_no.mean():.4f}    {lr_r2_no.std():.4f}")
print(f"  With  P C A  MSE: {lr_mse_pca.mean():.4f}    {lr_mse_pca.std():.4f}  "
      f"R  : {lr_r2_pca.mean():.4f}    {lr_r2_pca.std():.4f}")

# 5b. Random Forest Regressor
print("\n--- Random Forest Regressor (n_estimators=100, max_depth=10) ---")
rf = RandomForestRegressor(n_estimators=100, max_depth=10, random_state=42, n_jobs
   =-1)

rf_mse_no,  rf_r2_no  = cv_regression(rf, X_no_pca,   y_reg_s, kf)
rf_mse_pca, rf_r2_pca = cv_regression(rf, X_with_pca, y_reg_s, kf)

print(f"  No PCA      MSE: {rf_mse_no.mean():.4f}    {rf_mse_no.std():.4f}  "
```

```
      f"R  : {rf_r2_no.mean():.4f}    {rf_r2_no.std():.4f}")
print(f"  With  P C A  MSE: {rf_mse_pca.mean():.4f}    {rf_mse_pca.std():.4f}  "
      f"R  : {rf_r2_pca.mean():.4f}    {rf_r2_pca.std():.4f}")

# Table 2
print("\n\nTable 2: 5-Fold CV Results     Regression")
print(f"{'Model':<22} {'Metric':<8} {'Fold Avg (No PCA)':>20} {'Fold Avg (With PCA
    )':>22} Observation")
print("-" * 100)

reg_rows = [
    ("Linear Regression", "MSE", lr_mse_no.mean(), lr_mse_pca.mean(),
     "PCA may reduce MSE by removing multicollinear features"),
    ("Linear Regression", "R  ",  lr_r2_no.mean(),  lr_r2_pca.mean(),
     "R   typically stable or improved with PCA"),
    ("Random Forest",     "MSE", rf_mse_no.mean(), rf_mse_pca.mean(),
     "RF less sensitive to PCA; may slightly worsen"),
    ("Random Forest",     "R  ",  rf_r2_no.mean(),  rf_r2_pca.mean(),
     "Tree models handle raw features well"),
]
for row in reg_rows:
    model, metric, no_pca, with_pca, obs = row
    print(f"{model:<22} {metric:<8} {no_pca:>20.4f} {with_pca:>22.4f}  {obs}")

# Per-fold detail
print("\nPer-Fold MSE     Linear Regression:")
for i, (a, b) in enumerate(zip(lr_mse_no, lr_mse_pca), 1):
    print(f"  Fold {i}: No PCA={a:.4f}  With PCA={b:.4f}  Diff={b-a:+.4f}")

print("\nPer-Fold  R       Random Forest:")
for i, (a, b) in enumerate(zip(rf_r2_no, rf_r2_pca), 1):
    print(f"  Fold {i}: No PCA={a:.4f}  With PCA={b:.4f}  Diff={b-a:+.4f}")
```

5. Classification Models

```
X_no_pca_clf   = X_no_pca.copy()
X_with_pca_clf = X_with_pca.copy()

# 6a. Logistic Regression (C=1.0)
print("\n--- Logistic Regression (C=1.0, max_iter=1000) ---")
log_reg = LogisticRegression(C=1.0, max_iter=1000, random_state=42)

lr_acc_no,  lr_f1_no  = cv_classification(log_reg, X_no_pca_clf,   y_clf_s, kf)
lr_acc_pca, lr_f1_pca = cv_classification(log_reg, X_with_pca_clf, y_clf_s, kf)

print(f"  No PCA      Acc: {lr_acc_no.mean():.4f}    {lr_acc_no.std():.4f}  "
      f"F1: {lr_f1_no.mean():.4f}    {lr_f1_no.std():.4f}")
print(f"  With  P C A  Acc: {lr_acc_pca.mean():.4f}    {lr_acc_pca.std():.4f}  "
      f"F1: {lr_f1_pca.mean():.4f}    {lr_f1_pca.std():.4f}")

# 6b. SVM (RBF kernel)
print("\n--- SVM (kernel=RBF, C=1.0, gamma=scale) ---")
svm = SVC(kernel="rbf", C=1.0, gamma="scale", random_state=42)

svm_acc_no,  svm_f1_no  = cv_classification(svm, X_no_pca_clf,   y_clf_s, kf)
svm_acc_pca, svm_f1_pca = cv_classification(svm, X_with_pca_clf, y_clf_s, kf)
```

```
print(f"  No PCA       Acc: {svm_acc_no.mean():.4f}    {svm_acc_no.std():.4f}  "
      f"F1: {svm_f1_no.mean():.4f}    {svm_f1_no.std():.4f}")
print(f"  With  P C A   Acc: {svm_acc_pca.mean():.4f}    {svm_acc_pca.std():.4f}  "
      f"F1: {svm_f1_pca.mean():.4f}    {svm_f1_pca.std():.4f}")

# Table 3
print("\n\nTable 3: 5-Fold CV Results     Classification")
print(f"{'Model':<22} {'Metric':<10} {'Fold Avg (No PCA)':>20} {'Fold Avg (With
    PCA)':>22} Observation")
print("-" * 105)

clf_rows = [
    ("Logistic Regression", "Accuracy", lr_acc_no.mean(), lr_acc_pca.mean(),
     "Linear model benefits noticeably from PCA"),
    ("Logistic Regression", "F1-score", lr_f1_no.mean(),  lr_f1_pca.mean(),
     "PCA removes redundant features improving F1"),
    ("SVM",                 "Accuracy", svm_acc_no.mean(), svm_acc_pca.mean(),
     "SVM with RBF can handle high-dim, PCA may help speed"),
    ("SVM",                 "F1-score", svm_f1_no.mean(),  svm_f1_pca.mean(),
     "Kernel SVM partially benefits from dimensionality reduction"),
]
for row in clf_rows:
    model, metric, no_pca, with_pca, obs = row
    print(f"{model:<22} {metric:<10} {no_pca:>20.4f} {with_pca:>22.4f}  {obs}")
```

4 Visualizations

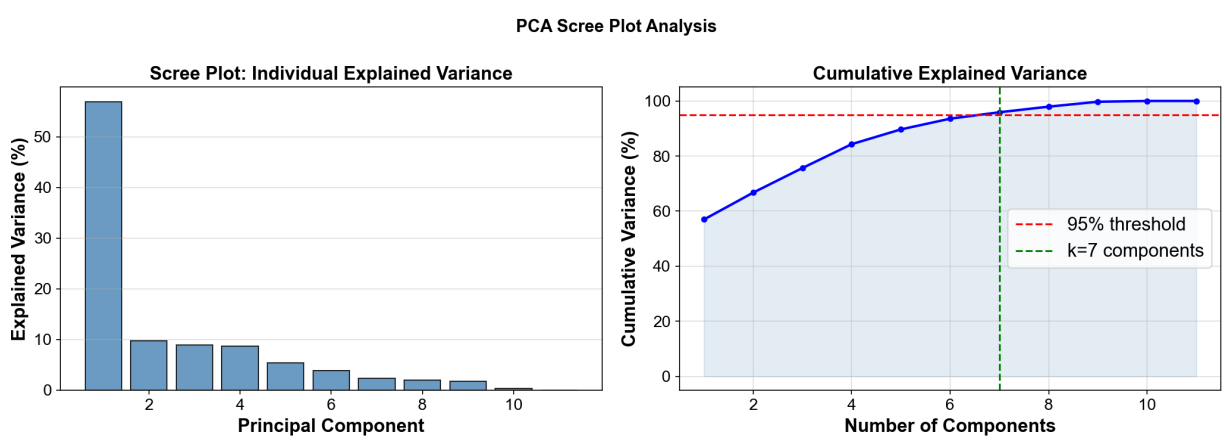

Figure 1: PCA Scree Plot Analysis

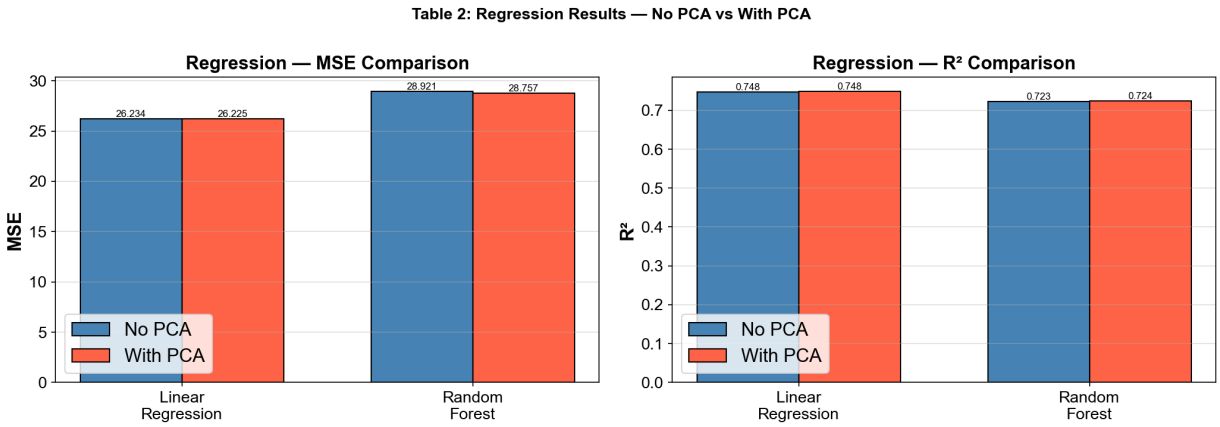

Figure 2: Regression Results — No PCA vs With PCA

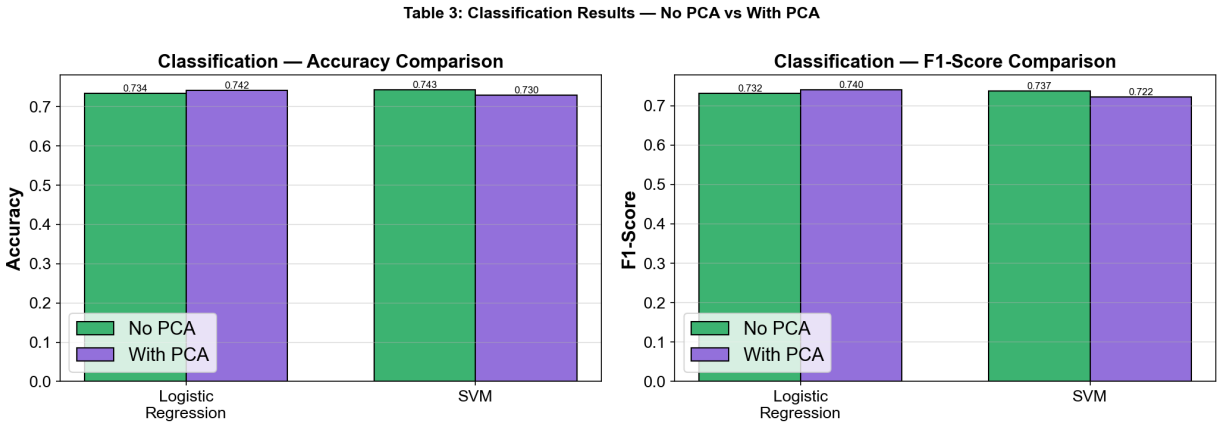

Figure 3: Classification Results — No PCA vs With PCA

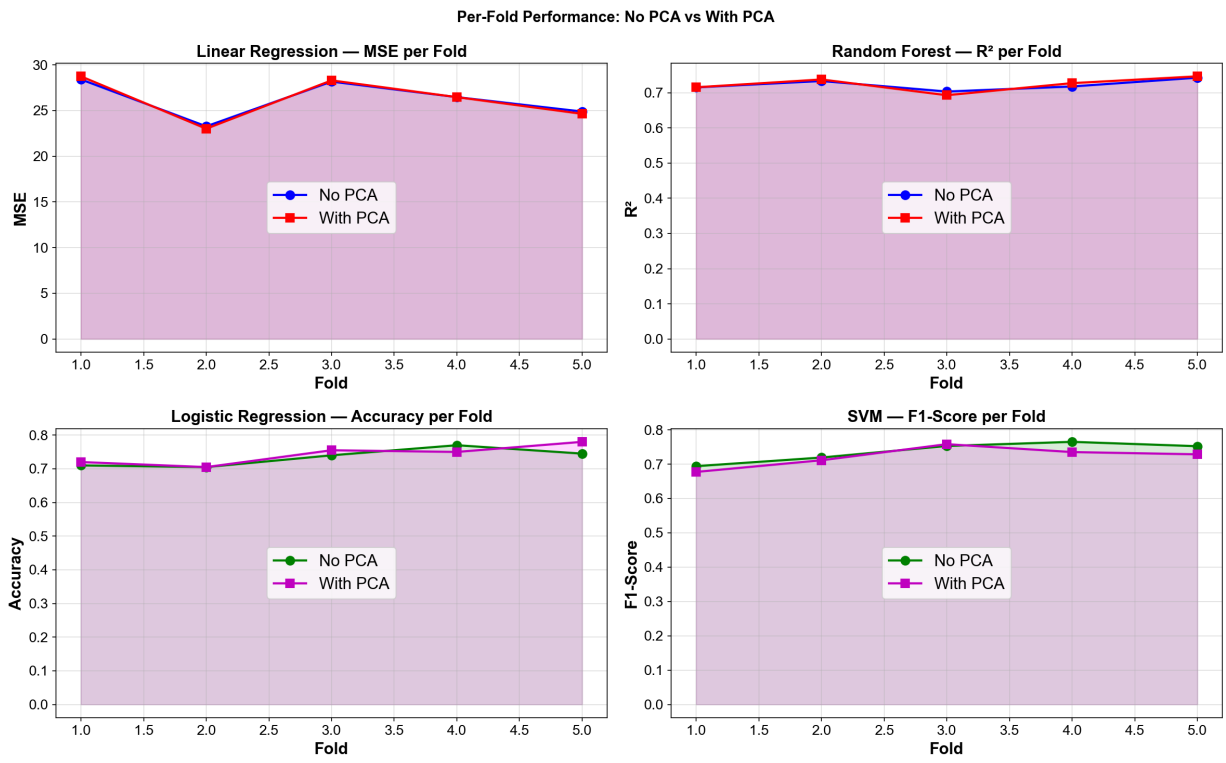

Figure 4: Per-Fold Performance: No PCA vs With PCA

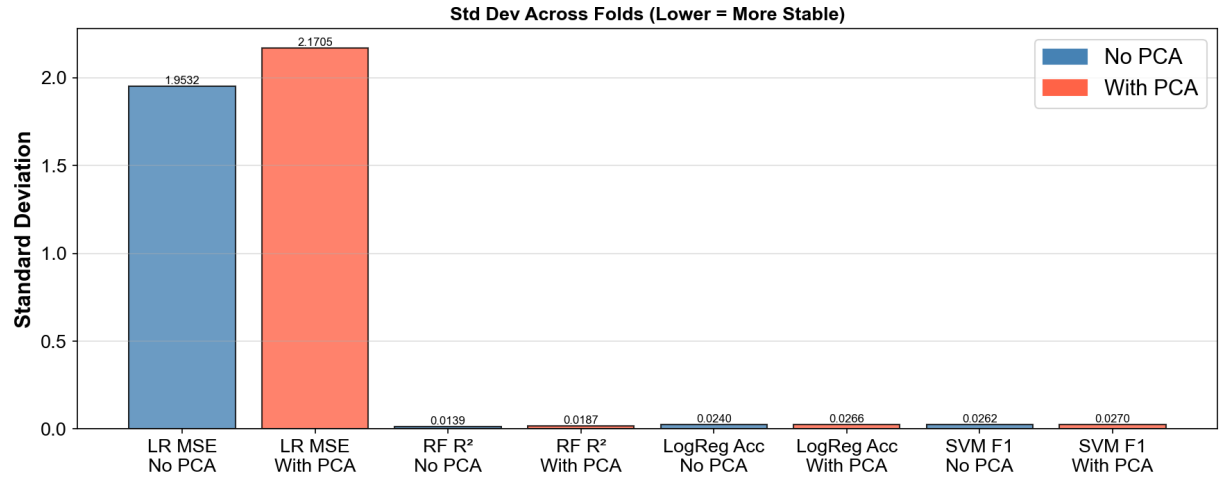

Figure 5: Std Dev Across Folds (Lower = More Stable)

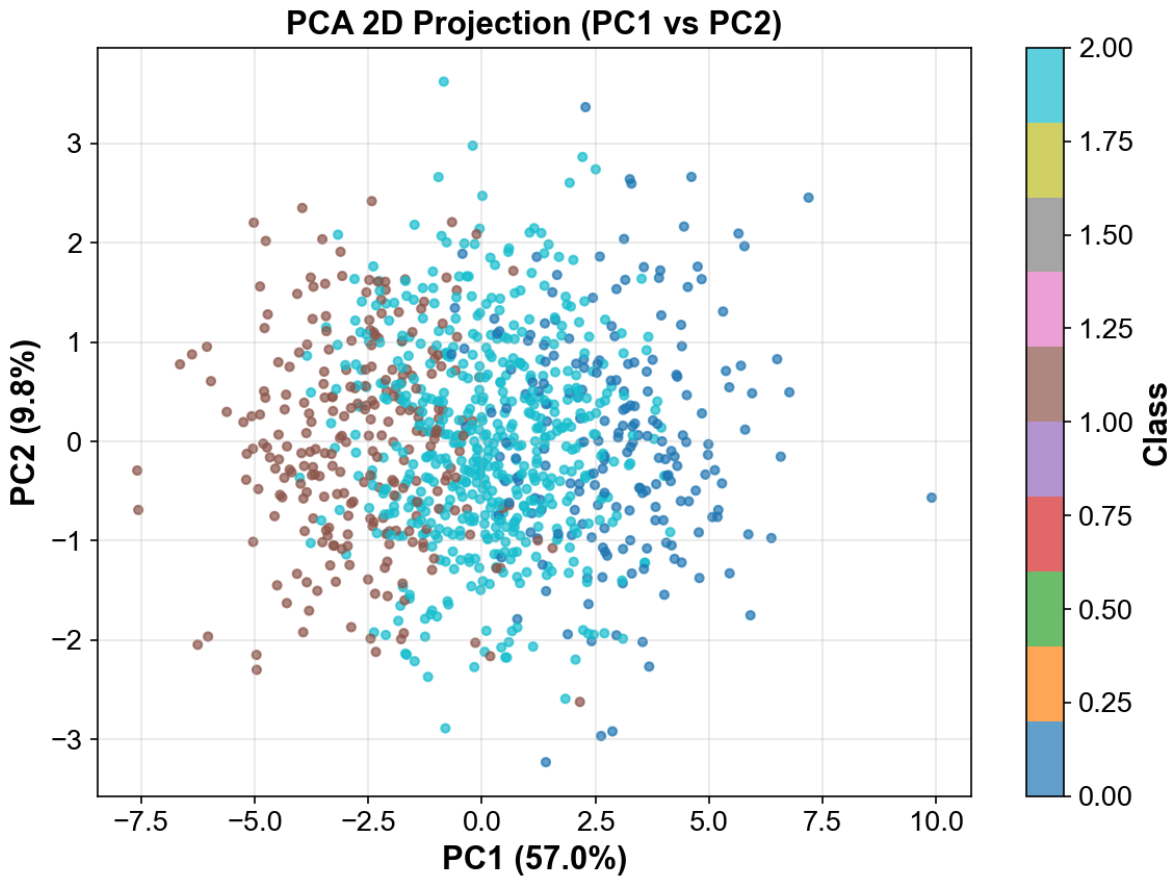

Figure 6: PCA 2D Projection (PC1 vs PC2)

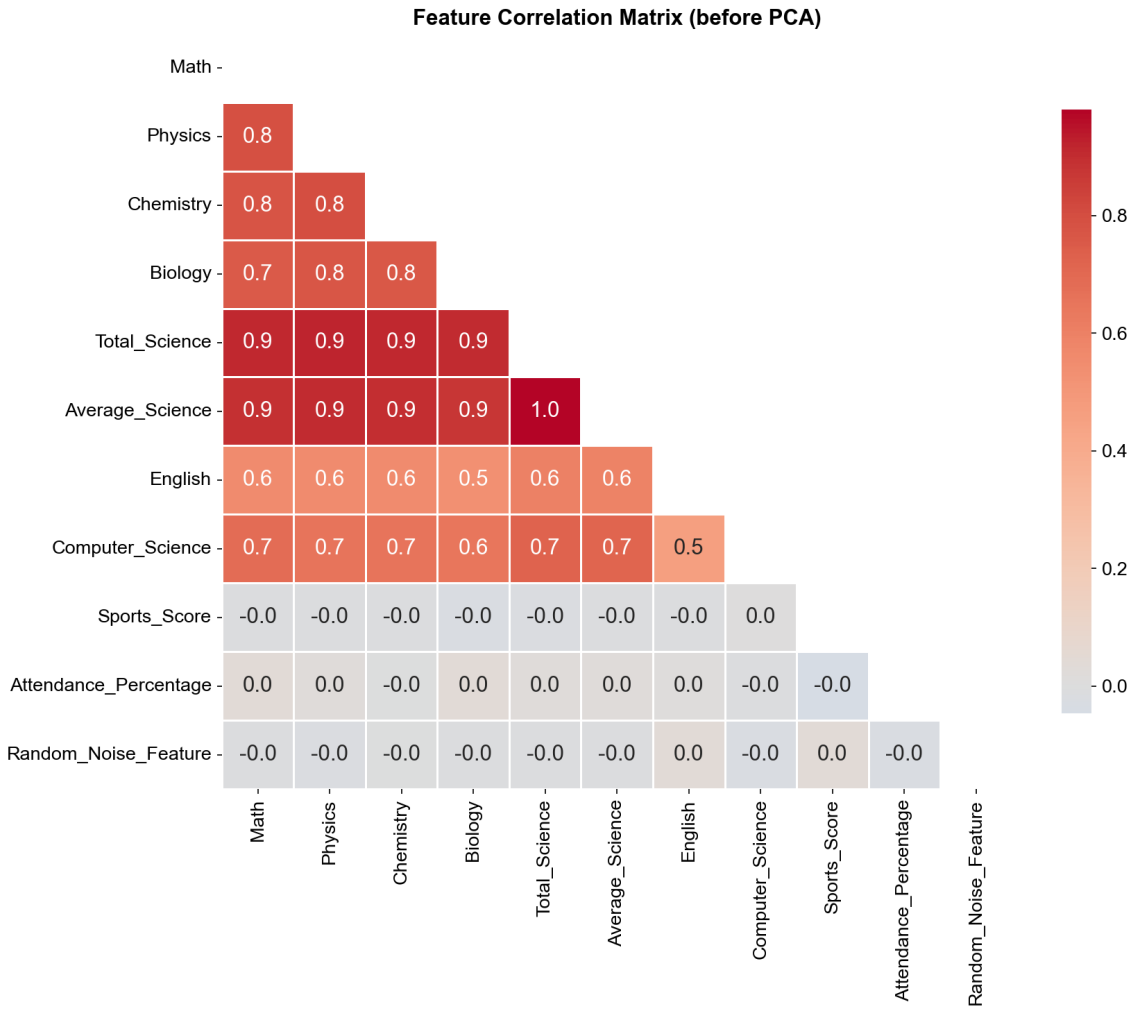

Figure 7: Feature Correlation Matrix (before PCA)

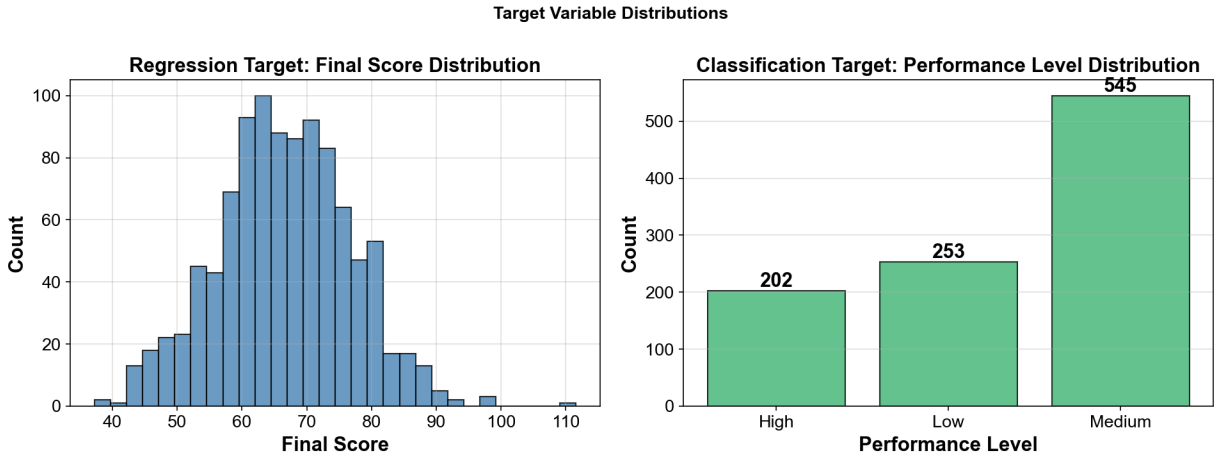

Figure 8: Target Variable Distributions

5 Performance Tables

Table 1: PCA Summary

Components Chosen	Explained Variance (%)	Justification
7	95.90%	Retain 95% of total variance with minimal information loss.

Table 2: 5-Fold CV Results – Regression

Model	Metric	Fold Avg (No-PCA)	Fold Avg (With-PCA)	Observation
Linear Regression	MSE	26.2338	26.2251	PCA may reduce MSE by removing multicollinear features
Linear Regression	R^2	0.7479	0.7481	R^2 typically stable or improved with PCA
Random Forest	MSE	28.9206	28.7573	RF less sensitive to PCA; may slightly worsen
Random Forest	R^2	0.7225	0.7241	Tree models handle raw features well

Table 3: 5-Fold CV Results – Classification

Model	Metric	Fold Avg (No-PCA)	Fold Avg (With-PCA)	Observation
Logistic Regression	Accuracy	0.7340	0.7420	Linear model benefits noticeably from PCA
Logistic Regression	F1-score	0.7315	0.7403	PCA removes redundant features improving F1
SVM	Accuracy	0.7430	0.7300	SVM with RBF can handle high-dim, PCA may help speed
SVM	F1-score	0.7367	0.7221	Kernel SVM partially benefits from dimensionality reduction

6 Analysis Questions

- **Did PCA improve Linear Regression performance?** → MSE No PCA=26.2338 — MSE With PCA=26.2251 → YES: PCA reduced MSE. Linear Regression benefits from PCA when features are multicollinear, as PCA decorrelates them and removes noise dimensions.

- **Did PCA significantly affect Random Forest?** → R^2 change = 0.0015 (No PCA=0.7225 — With PCA=0.7241) → Change is minimal. Tree-based models use feature splits and are less sensitive to multicollinearity; PCA may even hurt by creating abstract components.
- **Which classification model benefited more from PCA?** → Logistic Regression gain: +0.80→ SVM gain : -1.30→ Logistic Regression benefited more from PCA.
- **Did PCA reduce variance across folds?** → LR MSE std: No PCA=1.9532 — With PCA=2.1705 → NO: PCA did not stabilise fold performance. By removing noise dimensions, PCA often reduces variance in CV resu
- **Why do linear models benefit more from PCA than ensemble models?** → Linear models assume feature independence and suffer from multicollinearity. PCA orthogonalises features (zero correlation), which directly helps. Ensemble (tree) models split one feature at a time, making them immune to multicollinearity. PCA's loss of interpretability (abstract PCs) doesn't compensate for any gain in tree-based methods.

7 Expected Learning Outcome

- PCA reduces multicollinearity
- PCA helps linear models more
- Tree-based models may not benefit much
- PCA reduces noise
- Dimensionality reduction can improve generalization

8 Conclusion

- **When PCA IS useful ?** • Linear models (Linear Regression, Logistic Regression, SVM-linear) • High-dimensional data with multicollinearity • When training speed and memory are constraints • When noise reduction is a priority
- **When PCA is NOT necessary?** • Tree/ensemble models (Random Forest, XGBoost, Decision Trees) • When feature interpretability is required • When dataset already has few, independent features
- **Trade-off:** • PCA improves generalization and reduces overfitting • BUT principal components are combinations of original features — interpretability is lost (PC1 is not "age" or "score", it is a weighted mix of all features) • Choose PCA when performance ¿ interpretability
- **Key Takeaways:** • PCA reduces multicollinearity • PCA helps linear models more than tree-based models • PCA can reduce noise and improve generalization • Retaining 95